

Forge with Certified Symmetry

Stabilizer chains, canonical images,
symbolic orbit counting, and algebraic reduction

Prepared for Vladimir Reshetnikov

September 15, 2026

Abstract

Forge already proposes a wide range of certificate-producing mathematical workers. This article adds a coherent new capability: reasoning with a finite permutation group through a small, exact certificate, then reusing that certificate to canonicalize configurations, count orbits, and reduce symmetric algebraic searches. The principal object is a complete stabilizer chain whose strong generators retain straight-line provenance in the original generators. Its checker proves enough local facts to justify both positive and negative membership answers, an exact order, and a unique coset normal form. A separate coset-cover certificate establishes canonical images; recognized symmetric and alternating groups admit substantially cheaper image and counting certificates. A sparse Reynolds-projection certificate connects the same machinery to polynomial reasoning without enumerating the group.

The accompanying standard-library Python implementation was executed, with independent oracle comparisons, malformed-input controls, and search-free replay. It checked a certificate for $\mathfrak{S}_{40}$ using 820 transversal entries, not 40! explicit permutations. These are Python experiments and mathematical soundness arguments, not Lean-kernel proofs. The article identifies the exact reification and formalization obligations needed to make the extension part of a real **forge** tactic.

Inspected Forge revision	58ea206bd7ad501b930add568151217bcc7f82a2
Delivered	LaTeX and PDF; executable producers and separate checkers; recorded certificates; replay scripts; tests; integration specifications.
Not claimed	An implemented Lean tactic, a verified Python checker, or a speedup over grind .

Contents

1	What this adds, and what it deliberately does not repeat	3
1.1	The scope of the novelty audit	3
1.2	Why a coherent symmetry layer is more useful than another isolated solver	4
2	The mathematical contract	4
3	A complete stabilizer-chain certificate	5
3.1	Certificate representation	5
3.2	Sifting	6
3.3	Soundness, without trusting the chain constructor	6
3.4	A rejected partial-chain example	7
4	A deterministic certificate producer	7
5	Canonical images with checkable pruning	8
5.1	A lower bound for an entire coset	8
5.2	The coverage certificate	9
5.3	What this does and does not make easy	9
6	Recognized families: an exact escape from factorial search	10
6.1	Canonical images for the symmetric group	10
6.2	Canonical images for the alternating group	10
7	Exact orbit counting from cycle inventories	11
7.1	A formula valid for every number of colors	11
7.2	Fixed-weight counting	12
7.3	Counting huge recognized groups without enumerating them	12
7.4	The singleton-cycle adapter hazard	13
7.5	Constraints require a different fixed-point calculation	13
8	Reynolds projection without group enumeration	13
8.1	The orbit formula	14
8.2	An orbit certificate rather than a second orbit search	14
8.3	A precise reduction for Forge’s finite cone dictionary	15
8.4	Conditions that cannot be dropped	16
9	The source-level symmetry gate	16
9.1	A sufficient semantic interface	16

9.2	A tested narrow gate: unsigned variable permutations of CNF	16
9.3	How to propose source symmetries without trusting discovery	17
9.4	Dependent types, nominal identities, and context changes	17
10	Concrete Lean and library integration	18
10.1	A small set of worker requests	18
10.2	Proposed module decomposition	18
10.3	Reusable libraries and exact boundaries	19
10.4	The reification proof obligations are finite and explicit	20
11	Executed experiments and what they establish	20
11.1	Main case construction and oracle separation	21
11.2	Large certificates	21
11.3	Search-free replay	22
11.4	The supplementary Reynolds run	22
11.5	Regression tests, failures, and corrections	23
11.6	Claims not supported by this evidence	23
12	An implementation order with concrete acceptance gates	23
13	Further research that the certificate boundary makes possible	24
13.1	Shorter bases and proof compression	24
13.2	Better canonical bounds with local certificates	25
13.3	Conjugacy-class inventories with coverage proofs	25
13.4	Representation-theoretic algebraic compression	25
14	Conclusion	25
A	Using the delivered package	26
A.1	A direct API example	26
A.2	Source and result inventory	27
A.3	Building the article	27
B	Status at a glance	28
	References	29

1 What this adds, and what it deliberately does not repeat

The natural first proposals for a tactic stronger than `grind`—induction, auxiliary lemmas, nonlinear certificates, witness synthesis, and solver cooperation—are no longer useful contributions to this repository. Forge’s inspected revision already contains three merged rounds of proposals, a broad implementation-status ledger, and extensive certificate schemas. Its later rounds include exact closure constructions, noncommutative reasoning, analytic certificates, and quantitative probabilistic and adversarial workers [1, 2, 3]. Repeating those designs would expand the article without expanding the system.

The extension developed here starts from a different finite object:

$$G = \langle g_1, \dots, g_m \rangle \leq \mathfrak{S}_n,$$

and asks how a proof planner can use G *without enumerating its elements*. The answer is not simply “call GAP.” It is an explicit certificate language, a local checker, a proof of what acceptance implies, and several new consumers of the resulting group facts.

Existing prerequisite	Added object or algorithm	New consequence
Exact arithmetic and certificate binding	Original-generator word DAG plus complete stabilizer chain	Exact membership, nonmembership, group order, unique coset coordinates.
Finite search and checked coverage	Coset-cover canonical-image proof	A least orbit representative and an explicit transporter.
Exact quantities	Cycle inventory and family-recognition certificates	Symbolic coloring counts and fixed-weight orbit counts.
Polynomial cone workers	Locally checked monomial orbits and Reynolds projection	Invariant polynomial projection; a precise route to orbit-reduced cone searches.
Source-owned synthesis evidence	Generator-wise semantic symmetry gate	Safe representative restriction that respects exact hypotheses, dictionaries, and universes.

1.1 The scope of the novelty audit

The audit used the pinned Forge README, its complete implementation-status ledger, the contribution catalogue, the certificate-schema section, and the discussion of Leant and Djex [1, 2, 3, 4]. Leant’s README and current revision metadata, and Djex’s README, supplied the neighboring-project context [5, 6]. These inspected capability descriptions do not provide the certificate suite developed here.

This is a *scoped engineering novelty claim*, not a claim to have read every line of all 27 frozen submissions. Repository code search returned no result even for the positive-control term “Bernstein,” which is visibly present in the status ledger. Its negative results were therefore discarded as evidence of absence. The package’s `docs/NOVELTY.md` records this limitation so that later integration does not turn a bounded audit into an exhaustive claim.

Nor are Schreier–Sims, Burnside’s lemma, Reynolds averaging, or symmetry breaking new mathematics. They are established methods. GAP and SymPy provide permutation-group algorithms; Mathlib already exposes an orbit-counting theorem; certified symmetry-breaking systems and symmetry-adapted

polynomial optimization have their own literature [7, 8, 9, 12, 14]. The contribution is the particular certificate interfaces, their compositional soundness arguments, the tested independent replay paths, and the integration plan for Forge.

1.2 Why a coherent symmetry layer is more useful than another isolated solver

A group certificate can be amortized across many queries. The same checked G can answer whether a proposed renaming is allowed, produce a least representative, provide a counting denominator, and certify a monomial orbit. It is not tied to one formula or one coloring. The source-level authorization to *use* the group, however, remains tied to the exact obligation.

For example, a cyclic renaming might preserve a family of polynomial hypotheses, yet fail to preserve a distinguished coefficient in the goal. The group-theoretic certificate remains correct; the proposed proof transformation is not authorized. Separating these objects is the central design decision. It permits aggressive external search without allowing a correct calculation about the wrong group action to become a proof of the original theorem.

The initial production target should be concrete finite actions on **Fin n**. Arbitrary finitely presented groups, arbitrary actions on infinite types, and unrestricted discovery of semantic program equivalence are outside this extension. A representation of an abstract group by permutations also needs a separate faithfulness theorem before its image order can be called the abstract group's order.

2 The mathematical contract

Let $[n] = \{0, \dots, n-1\}$. Permutations are arrays of length n , and all arrays are checked to be bijections. Our composition convention is

$$(ab)(i) = a(b(i)), \quad (g \cdot c)(j) = c(g^{-1}(j)).$$

Thus the right factor acts first, and $g \cdot c$ moves the color at position i to position $g(i)$. This convention also defines the action on exponent vectors. It is tested with noncommuting permutations; a commuting test cannot detect reversal of multiplication.

The authoritative input is an ordered list of original generators, separate from the certificate. A certificate may contain derived permutations, but only through words in those original generators. Renumbering generators, changing the permutation degree, or replacing a source expression by another equal-looking expression is not a harmless metadata edit.

We use four different forms of conclusion.

Group fact.

A statement about exactly $G = \langle g_1, \dots, g_m \rangle$: membership, order, or a complete orbit.

Action fact.

A statement about the explicitly defined G -action: a transporter, a canonical image, or an orbit count.

Source authorization.

A proof that the action preserves the original hypotheses, target, or objective in the way required by a specific transformation.

Operational outcome.

Accepted certificate, invalid certificate, or resource exhaustion. Exhaustion is *unknown*; it is not nonmembership, infeasibility, or a refutation.

A proper subgroup of the true symmetry group is sufficient for safe symmetry reduction. It merely merges fewer cases. That same proper subgroup is *not* sufficient for claiming the number of orbits under the full symmetry group. Both computations can be internally correct while answering different questions.

3 A complete stabilizer-chain certificate

A stabilizer chain replaces a large set of group elements with a sequence of small orbit tables. The difficulty for a proof-producing tactic is not obtaining a plausible chain; it is checking that the chain is complete. An incomplete chain may reject an actual group member. Our acceptance condition therefore checks every required Schreier residue, not a random sample and not a producer-selected subset.

3.1 Certificate representation

The certificate has five fields:

```
{
  "format": "forge.symmetry.chain.v1",
  "word_dag": [ ... ],
  "strong": [ ... ],
  "levels": [ ... ],
  "order": ...
}
```

The degree and original generator arrays are not fields of this object. They are supplied independently by the caller.

The word DAG has operations `id`, `gen k`, `inv a`, and `mul a b`. Every child index must precede its parent. A `gen k` node refers to original generator g_k . Evaluating the DAG therefore proves that every evaluated node belongs to G . The strong-generator list contains references to these nodes. The checker requires distinct, nonidentity permutations and requires every nonidentity original generator to occur in the strong list. Thus the strong list generates exactly G , rather than merely some subgroup or overgroup.

The prototype uses the full base $0, 1, \dots, n-1$. This sacrifices an optimization but eliminates an additional proof obligation: there is no unverified assertion that a shorter base is faithful. A future partial-base format must certify that its final pointwise stabilizer is trivial.

Let S be the set of evaluated strong generators, and define

$$S_i = \{s \in S : s(0) = 0, \dots, s(i-1) = i-1\}, \quad K_i = \langle S_i \rangle.$$

At level i , the certificate supplies an orbit tree. Its root is i , with identity transversal. A nonroot record has the form

$$(x, \text{earlier parent index, signed strong-generator index}).$$

The checker reconstructs $t_x = s t_{\text{parent}}$, requires $s \in S_i^{\pm 1}$, and checks $t_x(i) = x$. The points must be distinct. This establishes that every listed point lies in the actual K_i -orbit of i , and every t_x belongs to K_i .

Denote the listed point set by Ω_i and its transversal set by T_i . The remaining checks are:

1. For every $s \in S_i^{\pm 1}$ and $x \in \Omega_i$, require $s(x) \in \Omega_i$.
2. Form the Schreier residue

$$r_{i,s,x} = t_{s(x)}^{-1} s t_x. \quad (1)$$

Require that it sifts to the identity through levels $i + 1, \dots, n - 1$.

3. Require that the claimed order is exactly $\prod_i |\Omega_i|$.

The last check alone has almost no force: a malformed collection of tables also has a product. The closure and residue checks are what make that product a group order.

3.2 Sifting

Given a permutation h fixing the first i base points, sifting at level i computes $x = h(i)$. If $x \notin \Omega_i$, it fails. Otherwise it replaces h by $t_x^{-1}h$, which fixes the first $i + 1$ points. After the full base, success requires the identity permutation. The code also validates query permutations before using them.

A successful sift yields a factorization

$$h = t_{x_i} t_{x_{i+1}} \cdots t_{x_{n-1}}. \quad (2)$$

This factorization is already a positive membership witness. A failed sift becomes a *negative* membership witness only after the chain's completeness has been established.

3.3 Soundness, without trusting the chain constructor

Lemma 3.1 (The listed orbit is exact). *For an accepted level i , $\Omega_i = K_i \cdot i$.*

Proof. The reconstructed tree gives $\Omega_i \subseteq K_i \cdot i$. The root belongs to Ω_i , and closure under every generator and inverse gives the opposite inclusion by induction on word length. $\square$

Lemma 3.2 (Stabilizer equality). *For every accepted level $i < n$,*

$$\text{Stab}_{K_i}(i) = K_{i+1}.$$

Proof. Every generator of K_{i+1} lies in K_i and fixes i , so one inclusion is immediate. For the other inclusion, first observe that a successful sift of a residue in (1) expresses it as a product of transversals from levels $j \geq i + 1$. Each such transversal belongs to $K_j \subseteq K_{i+1}$. Thus every residue lies in K_{i+1} .

For completeness, the usual Schreier argument can be written directly. If $s \in S_i^{\pm 1}$ and $x \in \Omega_i$, equation (1) rearranges to

$$s t_x = t_{s(x)} r_{i,s,x}.$$

Beginning with the identity transversal, repeatedly use this equation while reading a word in $S_i^{\pm 1}$. Induction on word length writes every $h \in K_i$ as $t_{h(i)}k$ with $k \in K_{i+1}$. If $h(i) = i$, the root transversal is the identity, hence $h \in K_{i+1}$. $\square$

Theorem 3.3 (Complete-chain theorem). *An accepted certificate yields a bijection*

$$T_0 \times T_1 \times \cdots \times T_{n-1} \longrightarrow G, \quad (t_0, \dots, t_{n-1}) \longmapsto t_0 \cdots t_{n-1}.$$

Consequently its order product is $|G|$, and sifting decides membership in G .

Proof. The strong list generates G , so $K_0 = G$. The preceding lemma identifies each next group with the corresponding stabilizer. The full-base bottom group K_n is trivial because every permutation fixing every point is the identity. Repeated coset decomposition gives surjectivity.

For uniqueness, compare two products. Their values at 0 select the same point of Ω_0 , hence the same stored transversal t_0 . Cancel it. Their residual values at 1 select the same next transversal. Continue to the full base. All digits agree. The bijection proves the order formula. If a member failed sifting, its unique coset digit would be absent from an exact orbit, a contradiction. Conversely, successful sifting is the explicit product (2) of known group elements. $\square$

The mathematical theorem does not certify the Python implementation. It specifies the theorem a reflected Lean checker must eventually prove. The standalone checker is intentionally small enough for this proof effort to be bounded and inspectable.

3.4 A rejected partial-chain example

Take $G = \langle (0\ 1\ 2), (0\ 1) \rangle = \mathfrak{S}_3$. Supply all three orbit points at level 0, but only the roots at levels 1 and 2. Both original generators are present, every level's listed orbit is closed under its listed stabilizer generators, and the table-size product is 3. Nevertheless the point stabilizer contains a nontrivial transposition that the lower tables cannot represent. A Schreier residue fails sifting, so the checker rejects the certificate.

This exact case is in the regression suite. It tests the essential logical gate after superficial order-product checks have already passed. Another control removes a point from a cyclic orbit and recomputes the claimed product; it is rejected for lack of orbit closure, not merely for a stale order field.

4 A deterministic certificate producer

The producer is an incremental Schreier–Sims construction, chosen for a simple termination argument rather than competitive performance with established computer algebra systems [7, 8]. It starts with the original nonidentity generators. For each level it builds the orbit tree under the currently available prefix-fixing generators and their inverses. It scans Schreier residues, beginning at the deepest level, and sifts each residue through the lower tables.

If a residue fails, the producer adds the remaining nonidentity permutation as a new strong generator, together with its original-generator word DAG. It then rebuilds the tables. If all residues succeed, the tables are exported.

```
strong := nonidentity original generators
repeat:
  tables := orbit_trees(strong, full_base)
  for level from n-1 down to 0:
    for signed generator s fixing earlier base points:
      for x in tables[level]:
        r := inverse(t[s(x)]) * s * t[x]
        residual := sift_from(r, level+1, tables)
        if residual != identity:
          append residual and its word to strong
          restart
  return complete chain certificate
```

Proposition 4.1 (Finite insertion bound). *Without resource caps, the producer inserts at most $n(n-1)/2$ additional strong generators.*

Proof. A nonidentity residual after sifting fixes all points before the level where its image is missing. Adding it therefore strictly enlarges the orbit of that level’s base point. Strong-generator sets only grow, so none of the level orbits shrinks. At level i , at most $n-i$ points can occur because earlier points are fixed. Starting from at least one point per level, the total possible strict orbit growth is at most $\sum_{i=0}^{n-1} (n-i-1) = n(n-1)/2$. $\square$

The producer interns evaluated permutations and keeps one word for each value. This extensional merging is an optimization in untrusted code: the checker reevaluates the retained words. A final backward traversal trims dead word nodes and preserves only ancestors of the strong-generator roots. Orbit transversals need not be stored as separate long words because their local tree edges reconstruct them.

Let $s = |S|$, W be the retained word-DAG size, and

$$E = \sum_i |S_i^{\pm 1}| |\Omega_i|$$

be an upper bound on checked Schreier edges. Evaluating words costs $O(Wn)$ elementary array operations. Reconstructing orbit trees costs $O(n \sum_i |\Omega_i|)$. Each residue check uses at most n permutation compositions, each $O(n)$, giving $O(En^2)$ overall. Since $E = O(sn^2)$, $O(Wn + sn^4)$ is a safe coarse bound for the mathematical array operations of the checker. Python validation, hashing, allocation, and integer bit costs add overhead; this is not a measured runtime model.

At most $n(n+1)/2$ transversal entries are needed for the full base. The claimed order has at most $\lceil \log_2(n!) \rceil$ bits. These bounds concern the representation, not a promise that every supported degree fits a particular execution budget. The implementation separately caps degree, word nodes, Schreier checks, and enumeration. Reaching a cap raises a resource-refusal result.

5 Canonical images with checkable pruning

For a coloring $c : [n] \rightarrow \mathbb{N}$, the canonical-image problem is

$$\text{can}_G(c) = \min_{g \in G}^{\text{lex}} g \cdot c.$$

A transporter g proves only that a proposed image is reachable. It does not prove that the image is least. The additional certificate is a complete cover of the group’s coset decomposition, with a locally checkable lower bound at every pruned node.

5.1 A lower bound for an entire coset

At level i of an accepted chain, let $O_i(k)$ be the orbit of point k under K_i . Define

$$m_i(k) = \min_{t \in O_i(k)} c(t).$$

The producer computes these point orbits with union-find. The checker recomputes them by graph traversal under the prefix-fixing generators. It does not reuse the producer’s orbit partition.

A node of the search represents the coset pK_i . Its proposed lower bound is the tuple

$$L(p, i) = p \cdot m_i. \quad (3)$$

Lemma 5.1 (Coset lower bound). *For every $h \in K_i$ and every coordinate j ,*

$$(ph \cdot c)(j) \geq L(p, i)(j).$$

In particular $L(p, i) \leq_{\text{lex}} ph \cdot c$.

Proof. Put $k = p^{-1}(j)$. Then $(ph \cdot c)(j) = c(h^{-1}(k))$, and $h^{-1}(k) \in O_i(k)$. The coordinatewise lower bound follows from the definition of m_i . A coordinatewise lower bound is also a lexicographic lower bound. $\square$

The minima at different coordinates need not be simultaneously attainable by one permutation. That is harmless: a lower bound may be an optimistic relaxation. It becomes unsound only if the implementation mistakenly treats the relaxed tuple as a reachable solution. Reachability is checked separately through the transporter.

5.2 The coverage certificate

The canonical-image certificate contains a claimed best tuple b , a transporter, and a tree whose nodes are either ["cut"] or ["split", children]. The checker starts at $(p, i) = (1, 0)$ and derives all child cosets itself.

At a cut it verifies $L(p, i) \geq_{\text{lex}} b$. At a split it requires exactly one child for each transversal in T_i , in the chain's fixed table order. The child associated with $t \in T_i$ represents $(pt)K_{i+1}$. Missing children are rejected. Splitting below the full base is rejected. A leaf at the bottom has an exact bound because K_n is trivial.

Theorem 5.2 (Canonical-cover soundness). *Suppose the chain is accepted, the supplied transporter belongs to G and sends c to b , and every node of the coverage certificate passes its check. Then $b = \text{can}_G(c)$.*

Proof. Theorem 3.3 makes each split a disjoint, exhaustive decomposition of its parent coset. Induction on the finite cover tree shows that every element of G lies below a cut. The lower-bound lemma shows that each corresponding image is at least b . The checked transporter gives an element attaining b , so b is the minimum. $\square$

The producer performs two passes. First it searches for an incumbent, visiting children in increasing lower-bound order. Then it constructs a cover for the final incumbent. Only the second tree is the minimality certificate. Search-node counts and proof-node counts are recorded separately because an efficient discovery order need not yield a small proof of completeness.

5.3 What this does and does not make easy

The certificate makes each pruning decision explicit, but it does not make generic canonical-image search polynomial. In the worst case the coset tree approaches explicit enumeration. Repeated colors can leave the pointwise lower bound far below every attainable tuple, so many branches survive. In the

recorded development run, the generic $\mathfrak{S}_{15}$ case with colors repeating 0, 1, 2 exhausted a 20,000-node budget. This result is preserved as *unknown*, not hidden in a success count.

Three improvements are concrete but not implemented here: stabilize already individualized color classes to tighten the remaining group; cache lower-bound data for repeated cosets with checked transport; and accept stronger block-orbit bounds that solve a smaller matching problem with its own certificate. Each improvement must leave the complete-cover rule unchanged or replace it with an explicitly proved covering theorem. An external canonical-labeling program's answer alone is not a minimality certificate.

For orbit equality, however, there is an immediate useful consequence. If checked transporters satisfy $g_c \cdot c = b$ and $g_d \cdot d = b$, then $(g_d^{-1}g_c) \cdot c = d$. Distinct checked canonical images prove that c and d lie in different G -orbits. The latter negative conclusion requires complete minimality certificates on both sides, unless a separate invariant already distinguishes them.

6 Recognized families: an exact escape from factorial search

A generic algorithm should not ignore a group it can identify exactly. A checked chain proves the order, so family recognition can be elementary rather than probabilistic.

Proposition 6.1 (Symmetric and alternating recognition). *Let $G \leq \mathfrak{S}_n$ have an accepted chain.*

1. *If $|G| = n!$, then $G = \mathfrak{S}_n$.*
2. *If $n \geq 2$, every original generator is even, and $2|G| = n!$, then $G = \mathfrak{A}_n$.*

Proof. A subgroup of a finite group with the same cardinality is the whole group. In the second case the sign homomorphism gives $G \subseteq \mathfrak{A}_n$, whose order is $n!/2$ for $n \geq 2$. $\square$

The certificate is just a family tag. The checker recomputes the order and parity conditions; a solver's family label has no authority. This differs deliberately from a probabilistic recognition shortcut. SymPy documents Monte Carlo symmetric/alternating recognition for some degrees; such routines are acceptable proposal sources but cannot replace these exact gates [8].

6.1 Canonical images for the symmetric group

For $\mathfrak{S}_n$, the least image is the sorted tuple. The producer builds an explicit sorting transporter by assigning each occurrence of a color to an unused destination of that color. The checker requires the proposed best tuple to equal the sorted source, checks transporter membership by the accepted chain, and checks its action.

This certificate has no coset cover. Its minimality proof is the usual exchange argument for sorting, to be formalized once. Its data size is linear in n , apart from the already shared chain certificate. The Python implementation uses comparison sorting and quadratic inversion counting in the alternating case; no linear-runtime claim is made.

6.2 Canonical images for the alternating group

Even permutations introduce a small but important distinction.

Theorem 6.2 (Alternating canonical image). *Let $n \geq 2$ and let a be the sorted version of c . If c contains a repeated color, then $\text{can}_{\mathfrak{A}_n}(c) = a$. If all colors are distinct, let ϵ be the parity of the unique sorting transporter. For $\epsilon = 0$ the minimum is a ; for $\epsilon = 1$ it is a with its last two entries exchanged.*

Proof. With a repeated color, a transposition of two equal-colored source positions fixes c . Composing an odd sorting transporter with this odd stabilizer element makes it even without changing its image. With distinct colors, the sorting transporter is unique. If it is even, sorting is feasible. Otherwise a feasible image must differ from the sorted tuple by an odd permutation. Among all nonsorted permutations of a strictly increasing tuple, the lexicographically smallest exchanges only the final two entries: any permutation first differing earlier is larger at that earlier coordinate. This final transposition is odd, so composing it with the odd sorting transporter gives an even transporter and the claimed minimum. $\square$

For instance, under $\mathfrak{A}_5$ the tuple $(1, 0, 2, 3, 4)$ has minimum $(0, 1, 2, 4, 3)$, not $(0, 1, 2, 3, 4)$. In contrast, a repeated-color tuple such as $(1, 0, 0, 2, 3)$ can reach its sorted image. Both branches are covered by tests against full group enumeration at small degrees.

This family-specific certificate resolves the recorded $\mathfrak{S}_{15}$ budget fixture, but it does not retroactively make the generic search successful. The artifact ledger keeps both facts: a generic refusal and a separate accepted shortcut.

7 Exact orbit counting from cycle inventories

Let $a_G(\lambda)$ count the elements of G whose cycle-length multiset is λ . Here *fixed points are cycles of length one*. Define the cycle-index polynomial

$$Z_G(z_1, \dots, z_n) = \frac{1}{|G|} \sum_{\lambda} a_G(\lambda) \prod_{\ell \in \lambda} z_{\ell}.$$

The certificate stores the integer inventory a_G and denominator $|G|$, not approximate rational coefficients. Its cycle partitions are sorted, multiplicities are positive integers, duplicate partitions are forbidden, and the multiplicities must sum to the independently checked group order.

Those syntactic checks are necessary but insufficient. The producer constructs the inventory by a Cayley-graph breadth-first enumeration from the original generators. The checker uses a genuinely different representation: it enumerates every unique transversal normal form from Theorem 3.3, recomputes its cycles, and compares the entire inventory. It never calls the producer’s group enumeration.

7.1 A formula valid for every number of colors

Burnside’s lemma gives

$$N_G(q) = \#([q]^{[n]}/G) = \frac{1}{|G|} \sum_{\lambda} a_G(\lambda) q^{|\lambda|}. \quad (4)$$

An assignment fixed by a permutation is constant on each of its cycles, so a permutation with k cycles has exactly q^k fixed assignments. The averaging theorem is already available in Mathlib as

```
MulAction.sum_card_fixedBy_eq_card_orbits_mul_card_group
```

in `Mathlib.GroupTheory.GroupAction.Quotient` [9]. This is a concrete library reuse point, not a request to refund finite group actions.

A future Lean implementation should prove the integer form

$$|G| N_G(q) = \sum_{\lambda} a_G(\lambda) q^{|\lambda|},$$

then discharge exact division using $|G| > 0$. The Python checker verifies divisibility for each evaluated q . The mathematical derivation establishes why one verified inventory supports a theorem quantified over q ; executing finitely many values of q alone would not establish that universal theorem.

Example 7.1 (Ten positions on a circle). Let C_{10} act by rotation and D_{10} by rotations and reflections of ten positions. The actions are explicit permutations; $|C_{10}| = 10$ and $|D_{10}| = 20$. Their orbit-count formulas are

$$N_{C_{10}}(q) = \frac{q^{10} + q^5 + 4q^2 + 4q}{10},$$

$$N_{D_{10}}(q) = \frac{q^{10} + 5q^6 + 6q^5 + 4q^2 + 4q}{20}.$$

For two colors these give 108 necklaces and 78 bracelets. For three colors the bracelet count is 3,210. The D_{10} numerator is checked by the regression suite, rather than inserted as an unexplained formula.

7.2 Fixed-weight counting

For binary colorings with exactly k ones, a fixed assignment selects whole cycles. Therefore

$$N_{G,k} = \frac{1}{|G|} \sum_{\lambda} a_G(\lambda) [t^k] \prod_{\ell \in \lambda} (1 + t^{\ell}). \quad (5)$$

The checker computes each truncated product by an integer dynamic program, updating coefficients in descending order so that a cycle is selected at most once. This detail matters: an ascending in-place update would accidentally permit multiple selections of the same cycle.

For D_{10} and $k = 5$, the result is 16. The main experiment compares all tested weights with an independent coloring-orbit oracle; weights above n return zero. The empty action at $n = 0$ has one empty coloring even when $q = 0$, so the implementation treats $0^0 = 1$ in precisely that combinatorial context.

7.3 Counting huge recognized groups without enumerating them

For $n > 0$,

$$N_{\mathfrak{S}_n}(q) = \binom{q + n - 1}{n}, \quad (6)$$

with value zero when $q = 0$. Each orbit is a multiset of n colors. The $n = 0$ value is one.

For $n \geq 2$,

$$N_{\mathfrak{A}_n}(q) = \binom{q + n - 1}{n} + \binom{q}{n}. \quad (7)$$

Here $\binom{q}{n} = 0$ when $q < n$. To prove the additional term, examine a symmetric-group orbit. If a color repeats, its stabilizer contains an odd transposition, and the orbit remains one alternating-group

orbit. If every color is distinct, its stabilizer is trivial and its symmetric-group orbit splits into two alternating-group orbits. There are $\binom{q}{n}$ such all-distinct color sets. This also explains the parity distinction in Theorem 6.2.

Thus a recognized $\mathfrak{S}_{40}$ certificate immediately supports 41 binary-color orbits and 861 three-color orbits. No cycle inventory of $40!$ elements is constructed. The general inventory worker remains bounded by explicit group enumeration; the family formulas are a distinct certified route around that limitation.

7.4 The singleton-cycle adapter hazard

Mathlib’s `Equiv.Perm.cycleType` omits fixed points: its entries are at least two and its sum is the support cardinality. This is visible in the documented theorems `two_le_of_mem_cycleType` and `sum_cycleType` [10]. Our cycle inventory includes fixed points. Consequently a direct translation using only the cardinality of `cycleType` would undercount cycles, including giving zero cycles for the identity.

The adapter must reconstruct the full multiset by adjoining $n - |\text{supp}(g)|$ copies of one. Equivalently, the exponent in (4) is

$$\# \text{cycleType}(g) + n - |\text{supp}(g)|.$$

This is an example of a small representation mismatch that can invalidate every otherwise correct counting certificate.

7.5 Constraints require a different fixed-point calculation

Equation (4) counts all colorings, not satisfying assignments of an arbitrary CNF and not proper graph colorings. For a G -invariant constraint P , the correct numerator is

$$\sum_{g \in G} \#\{c : P(c) \text{ and } g \cdot c = c\}.$$

A proposed extension substitutes one variable per permutation cycle into P and invokes a separate exact finite counting worker. The substitution’s equivalence and each fixed-point count need certificates. This constrained-counting compiler is designed only; the delivered counting implementation covers unrestricted colors and fixed binary weight.

8 Reynolds projection without group enumeration

The symmetry layer can improve Forge’s existing polynomial workers instead of only adding a new combinatorics sublanguage. Over $\mathbb{Q}[x_0, \dots, x_{n-1}]$, the Reynolds operator is

$$\mathcal{R}_G(p) = \frac{1}{|G|} \sum_{g \in G} g \cdot p.$$

It is a linear projection onto invariant polynomials. Symmetry reduction in polynomial optimization has substantial established theory, including representation-theoretic reductions much stronger than the narrow construction below [14]. Our immediate target is an exact sparse certificate that fits Forge’s existing coefficient-based checkers, not a new numerical semidefinite solver.

8.1 The orbit formula

Let $p = \sum_e a_e x^e$, with unlisted coefficients taken as zero. The action on exponent vectors is $(g \cdot e)_j = e_{g^{-1}(j)}$. For a monomial orbit O ,

$$\mathcal{R}_G\left(\sum_{e \in O} a_e x^e\right) = \frac{\sum_{e \in O} a_e}{|O|} \sum_{e \in O} x^e. \quad (8)$$

Indeed each source monomial visits every destination in its orbit the same number $|G|/|O|$ of times. That equal-fiber argument removes the group order from the computation: only the monomial orbit must be enumerated.

8.2 An orbit certificate rather than a second orbit search

For each orbit intersecting the source support, the certificate gives a rooted tree of exponent vectors. The root must occur in the original support. A nonroot vector is linked to an earlier vector by an original generator index. The checker verifies every edge by an explicit coordinate permutation, requires all vectors to be distinct, requires different listed orbits to be disjoint, and checks closure under every original generator.

Because the set is finite and each generator acts bijectively, forward closure implies inverse closure. The tree proves reachability; closure proves that no point of the orbit was omitted. Finally, every source-support monomial must appear in one of the listed orbits. Thus the listed sets are exactly the relevant monomial orbits.

The checker independently reconstructs the average coefficient in (8). Coefficients are reduced integer numerator/positive-denominator pairs. Zero output coefficients are omitted, but an orbit whose average is zero still needs its coverage certificate: cancellation does not authorize ignoring source terms. The supplied output polynomial must agree exactly with the reconstructed result.

The monomial-orbit theorem needs only validated generator bijections, not a group-order computation. The prototype accepts an already checked chain for API consistency; a production adapter can expose a weaker validated-generator object and skip chain construction when projection is the only requested operation. This is an important dependency optimization, not a weakening of the orbit-closure checks.

Theorem 8.1 (Sparse projection soundness). *If the monomial-orbit certificate and its output check pass, the output is $\mathcal{R}_G(p)$. In particular it is G -invariant, and applying the operator again leaves it unchanged.*

Proof. Reachability and closure identify each listed component with its exact orbit. Disjointness and full source-support coverage partition all nonzero input terms into those components. Formula (8) computes the projection on each component. Adding the resulting components proves the claim. Each output has a constant coefficient on its orbit, which proves invariance and idempotence. $\square$

Example 8.2 (A cubic over forty variables). For the natural action of $\mathfrak{S}_{40}$,

$$\mathcal{R}_{\mathfrak{S}_{40}}(x_0^2 x_1) = \frac{1}{1560} \sum_{\substack{0 \leq i, j < 40 \\ i \neq j}} x_i^2 x_j.$$

The exponent vector has one distinguished squared coordinate and one distinct linear coordinate, so its orbit has $40 \cdot 39 = 1560$ elements. The delivered producer constructs that orbit tree, and the separate checker verifies its coverage and all $1/1560$ coefficients. It does not enumerate $\mathfrak{S}_{40}$.

This is not a magical cure for large polynomial supports. A degree- d monomial orbit can be large, and the number of degree- d monomials is $\binom{n+d-1}{d}$. The worker caps the number of orbit vectors. It returns unknown on resource exhaustion.

8.3 A precise reduction for Forge’s finite cone dictionary

Suppose an existing Forge worker seeks a certificate

$$p = \sum_{j \in J} c_j \phi_j, \quad c_j \in \mathbb{Q}_{\geq 0}, \quad (9)$$

where each ϕ_j has an existing proof of nonnegativity on the current domain. Assume the group permutes the *indexed* dictionary:

$$g \cdot \phi_j = \phi_{g \cdot j},$$

and p is invariant. Indexing matters: equal polynomials may carry different hypothesis receipts, so a coefficient dictionary must not identify them merely by display text.

For each dictionary orbit $O \subseteq J$, define $B_O = \sum_{j \in O} \phi_j$.

Theorem 8.3 (Orbit-reduced cone completeness). *Under these assumptions, (9) has a rational nonnegative solution if and only if*

$$p = \sum_{O \in J/G} d_O B_O, \quad d_O \in \mathbb{Q}_{\geq 0} \quad (10)$$

has one.

Proof. Given (9), average its images under G . The left side remains p . On an orbit O , the average coefficient is $d_O = |O|^{-1} \sum_{j \in O} c_j$, a nonnegative rational number. This yields (10). Conversely, expand each B_O and assign coefficient d_O to every member of O . $\square$

The corresponding algorithm is explicit. Validate generator permutations of dictionary indices and their polynomial identities. Compute exact dictionary orbits. Sum each orbit column using sparse rational polynomial addition. Pass the smaller column matrix to the existing cone-search engine. Expand an accepted reduced solution back to the original dictionary and let Forge’s existing cone checker validate the original target identity. The new reduction is an optional search accelerator, not a change to that checker’s trust boundary.

For a fully symmetric dictionary of $(x_i - x_j)^2$ with $i < j$, there is one dictionary orbit. At $n = 40$, this reduces 780 coefficient variables to one. The identity

$$40 \sum_i x_i^2 - \left(\sum_i x_i \right)^2 = \sum_{i < j} (x_i - x_j)^2$$

is a transparent example. The dimension reduction is a mathematical consequence of the dictionary action; no LP timing or Lean success-rate claim is inferred from it. The Reynolds projector is implemented and tested, but the adapter to Forge’s existing cone package is not implemented in this artifact.

8.4 Conditions that cannot be dropped

An invariant target is essential for the equivalence theorem. A dictionary closed only approximately under generators does not qualify. If coefficients are tied to hypotheses, the action must transport those hypothesis proofs. For domains with distinguished bounds such as $x_0 \geq 2$ and $x_i \geq 0$ for $i > 0$, the full symmetric action is generally unavailable.

Most importantly, $\mathcal{R}_G(p) \geq 0$ does not imply $p \geq 0$. For example, swapping two variables sends $p = x_0 - x_1$ to $-p$, so its average is zero while p takes negative values. The legitimate directions are preservation of an already proved nonnegativity statement on an invariant domain, or the invariant-target cone equivalence above. A tactic must not silently replace an arbitrary goal by the easier averaged goal.

9 The source-level symmetry gate

A correct permutation calculation can still justify an incorrect proof transformation. The missing ingredient is a proof that the permutations act on the *right source problem*. Leant and Djex already emphasize candidate-owned source evidence, exact type selections, and independent reconstruction; Forge has already adopted much of that boundary discipline [5, 6, 4]. It is inherited infrastructure, not a new contribution of this article. The new task is to specialize it to symmetry.

9.1 A sufficient semantic interface

For a source domain X , action $G \curvearrowright X$, and predicate P , the basic authorization is

$$\forall g \in G \forall x \in X, \quad P(g \cdot x) \leftrightarrow P(x). \quad (11)$$

It is sufficient to provide this equivalence for the original generators: it extends to inverses by substituting $g^{-1} \cdot x$, and to products by composition. If the task involves minimization, provide additionally an objective f with $f(g \cdot x) = f(x)$. A domain predicate is included in P , rather than silently assumed to be invariant.

Suppose a checked canonical selector r satisfies $r(x) \in G \cdot x$ and $r(r(x)) = r(x)$. Then

$$\exists x P(x) \iff \exists x [P(x) \wedge r(x) = x], \quad (12)$$

$$\forall x P(x) \iff \forall x [r(x) = x \Rightarrow P(x)]. \quad (13)$$

For the forward direction of (12), replace a witness by its representative and use invariance. For the reverse direction of (13), apply the representative-only theorem to $r(x)$ and transport the conclusion back. The other directions are immediate.

These formulas explain exactly what a symmetry tactic may do. In general it may *not* prove $x = r(x)$, replace every occurrence of x by $r(x)$ inside an arbitrary expression, or conjoin a lex-leader constraint to the original theorem as though that constraint followed pointwise. Symmetry breaking preserves existence or a properly transported universal claim; it need not preserve the set of individual solutions.

9.2 A tested narrow gate: unsigned variable permutations of CNF

The delivered gate accepts a concrete conjunctive normal form with one-based signed literals and optional exact integer objective weights. For each original generator it renames variable indices,

preserves literal polarity, normalizes each clause as a set of literals, normalizes the clause collection as a set of clauses, and requires exact equality with the original normalized formula. For an objective $\sum_i w_i x_i$, it additionally requires $w_{g(i)} = w_i$.

This is a sufficient syntactic automorphism test, not a complete semantic equivalence procedure. It correctly ignores duplicate clauses and duplicate literals, but it does not discover every symmetry hidden by logical equivalences. Signed variable flips, pseudo-Boolean constraints, arbitrary arithmetic atoms, and dependent Lean terms are not part of this implemented gate.

The test corpus uses ring clauses $(x_i \vee x_{i+1})$ with uniform weights. Dihedral generators preserve them. Appending a distinguished unit clause x_0 , or changing only the first objective coefficient, invalidates that authorization. Five positive cases and ten such negative cases are recorded. No SAT-solving performance experiment is implied by these gate tests.

Certified dominance and symmetry-breaking systems show that solver-side pruning can be proof logged, including transformations not justified by ordinary clause implication [12, 13]. A future Forge Boolean adapter can use those logs as proposals. It still needs a reconstruction or verified-checker bridge into Lean; merely running VeriPB externally is not Lean-kernel acceptance.

9.3 How to propose source symmetries without trusting discovery

For finite formulas or polynomial dictionaries, construct a colored incidence graph. Variable vertices carry sorts and admissible coordinate roles; operator vertices carry operator identity and arity; hypothesis vertices carry their exact class and any fixed source dependencies. Ordinary argument positions are ordered, so their edges have position colors. Only operators with a proved commutative or unordered interpretation may omit those position distinctions.

An automorphism finder such as nauty or Traces can propose generator permutations of the selected coordinate vertices [11]. The adapter then decodes each permutation and independently proves the corresponding source invariance equation. A graph automorphism is a search hint, not the source proof. Missing automorphisms only weaken the reduction; falsely authorized automorphisms can make it unsound.

When the user explicitly supplies a permutation action, discovery is unnecessary. The highest-value first frontend is therefore explicit: accept a finite coordinate map and a small list of generator-equivariance lemmas. Automatic graph-based discovery can be added after this source-preserving route is tested.

9.4 Dependent types, nominal identities, and context changes

Two local variables having the same printed type does not automatically make their exchange a symmetry. A later hypothesis may distinguish them. A dictionary may carry computational data. A selected universe may constrain which polymorphic instantiation is legal. Leant and Djex preserve these distinctions in their supported synthesis fragments [5, 6].

A production source inventory should make every local constant rigid by default. Only explicitly reified coordinates are eligible for permutation. The authorization must retain the exact declaration identity, universe instantiation, binder dependencies, selected typeclass instances, and local-hypothesis proofs. A source graph with different nominal dictionaries must not become the same graph merely because both dictionaries have the same type.

There is also a state-transition rule. If search adds a new hypothesis H , the previous group may cease to preserve the obligation. The controller may keep the old group-theoretic certificate, but it must

either prove generator-wise preservation of H or switch to a subgroup that preserves it. Keeping only generators that individually preserve H is safe, although it can miss useful products of rejected generators. A more complete stabilizer search is a separate optimization and must export its own original-generator membership evidence.

Cache keys therefore have two layers. A group cache uses the degree and exact original permutation arrays. An authorization cache uses the source obligation and the exact invariance proofs. Equality of the first key does not imply reusability of the second. Transporter and orbit caches also include the action and source object, not just its display string.

10 Concrete Lean and library integration

10.1 A small set of worker requests

The following is a proposed protocol, not existing `forge` syntax:

```
checkGroup(degree, originalGenerators, chainCertificate)
member(chckedGroup, candidatePermutation)
canonicalImage(chckedAction, sourceColors, imageCertificate)
countOrbits(chckedAction, cycleInventory | familyCertificate)
projectInvariant(chckedGenerators, sourcePolynomial, orbitCertificate)
reduceCone(chckedDictionaryAction, invariantTarget)
```

The returned evidence should identify its mathematical conclusion. For example, an image packet carries both a transporter statement and a least-image statement. A counting packet identifies the exact quotient relation. An invariant-projection packet states an equality with the Reynolds operator, not nonnegativity of the original polynomial.

The controller initially tries the inexpensive recognition branches: an already cached chain, an exact symmetric/alternating family gate, or a bounded orbit computation. It chooses a generic coset cover only when its budget permits. It never interprets an external timeout as a negative mathematical result.

10.2 Proposed module decomposition

These paths are proposed additions, not files claimed to exist in the inspected repository.

Module	Implementation and proof obligation
<code>Forge/Symmetry/PermArray.lean</code>	Decode bounded arrays into <code>Equiv.Perm (Fin n)</code> ; prove executable composition, inverse, and action agree with their semantic counterparts.
<code>Forge/Symmetry/WordDAG.lean</code>	Total, fuel-bounded DAG evaluator; prove that every evaluated node lies in the closure of the original generators.
<code>Forge/Symmetry/Chain.lean</code>	Decode orbit trees; compute all residue checks; prove Theorem 3.3, including the bijection of transversal digits and the negative-membership implication.
<code>Forge/Symmetry/Image.lean</code>	Compute point-orbit minima; check the complete coset cover; prove Theorem 5.2; expose the explicit transporter.

Module	Implementation and proof obligation
<code>Forge/Symmetry/Families.lean</code>	Exact order/parity recognition; prove the two canonical-image formulas and the symbolic coloring counts.
<code>Forge/Symmetry/Counting.lean</code>	Enumerate normal forms only below a cap; validate full cycle partitions; combine the fixed-coloring bijection with Mathlib’s Burnside theorem.
<code>Forge/Symmetry/Reynolds.lean</code>	Check connected closed monomial components; prove the orbit-average formula; expose sparse polynomial identities and the cone-reduction theorem.
<code>Forge/Symmetry/Reify.lean</code>	Preserve exact source coordinates and semantic generator lemmas; bind returned mathematical theorems to the original goal.

Do not begin by formalizing the optimized external search. Begin with the deterministic, bounded decoders and the mathematical acceptance theorems. A search program may then be replaced by GAP, SymPy, or a custom heuristic without changing the theorem it must satisfy.

The decoded certificate should be finite data, and its checker should be a total Lean function. One-time soundness theorems turn checked Boolean facts into propositions. Reflection still requires the source adapter to prove the reified statement is the intended source statement. A theorem about arrays of natural numbers is not automatically a theorem about the user’s matrices, local variables, or finite subtype.

The trust controls already described in Forge’s neighboring-project review—including completed checking rather than mere declaration visibility—remain prerequisites [4]. This proposal neither replaces that policy nor claims that a Python `VerifiedChain` object enforces it. That class is an ordinary mutable Python result object, not an unforgeable proof authority.

10.3 Reusable libraries and exact boundaries

Mathlib. Use `Equiv.Perm`, subgroup closure, stabilizers, finite group actions, cosets, and finite cardinalities. Inspected families under `Mathlib.GroupTheory` include `Perm`, `GroupAction`, and `SpecificGroups`. The Burnside theorem and cycle-type facts named in Section 7 were verified in the public generated documentation [9, 10]. The generated documentation is a rolling snapshot; these API names must be rechecked against the exact Mathlib revision selected for implementation. These are concrete mathematical building blocks; no claim is made that a ready-to-use reflected Schreier–Sims checker was found.

For the polynomial branch, use the source polynomial representation already selected by Forge and prove compatibility with the finite-variable renaming action. A direct Mathlib implementation can use multivariate polynomials over $\mathbb{Q}$, but introducing a second incompatible sparse format solely for this extension would be a poor integration choice. The prototype format is deliberately small, not a demand to replace Forge’s existing format.

GAP. GAP’s permutation-group facilities are a natural high-performance proposal backend [7]. Export transversals, strong generators, and original-generator provenance into the certificate schema. GAP’s internal chain format is not the interchange format: the adapter must prove nothing on GAP’s word alone. In particular, a purported derived generator with no original-generator word must not be accepted simply because the external system labels it “strong.”

SymPy. Use `schreier_sims` for chain construction. Transversals are available through `basic_transversals`. Original-generator words can be requested with `generator_product` using `original=True` [8]. Generator-product ordering and multiplication conventions require explicit tests. In the installed SymPy 1.14.0, the supplementary test confirmed

$$\text{our mul}(a, b) = \text{SymPy Permutation}(b) * \text{Permutation}(a)$$

for a noncommuting fixture. A future adapter must also expand any returned inverse factor through its original provenance rather than confuse inverse generators with unrelated source inputs.

The optional external-oracle script compares seven large group orders and 140 membership answers with SymPy. This is corroboration by another implementation, not a formally verified oracle and not a performance comparison. SymPy is not imported by the delivered core producer, core checker, or standalone replay.

nauty/Traces and certified Boolean solvers. Use `nauty/Traces` for candidate automorphisms of a carefully colored graph; use generator-wise semantic replay to authorize them [11]. For Boolean search, certified dominance or symmetry-breaking logs can provide additional proof structure [12, 13]. Neither a canonical-labeling answer nor an external proof checker exit code is a Lean theorem without a bridge. These backends were researched but not installed or tested as part of this artifact.

10.4 The reification proof obligations are finite and explicit

For a first concrete frontend, the adapter must supply:

1. A bijection between the source coordinate type and `Fin n`, or an explicit statement that only the represented image action is being analyzed.
2. Exact interpretations of the original generator arrays as source permutations.
3. Generator-wise action compatibility with source colorings, monomials, or dictionary columns.
4. The source-preservation lemmas needed for the requested consequence: invariance, objective preservation, or quotient identification.
5. A kernel-checked application of the certificate soundness theorem to the original obligation, under the repository’s axiom and verification policy.

Failure at any item produces an unsupported-fragment or remaining-obligation result. The remaining obligations should be available to Forge’s existing local reasoning and synthesis workers. They are not to be deleted because the group computation succeeded.

11 Executed experiments and what they establish

All numbers in this section come from the delivered result files. The main run used Python 3.13.5 on Linux, seed 20260915, and no third-party libraries. Its recorded elapsed time was approximately 15.68 seconds. This is one authoring-environment observation, not a portable runtime promise or a statistical benchmark. The optional external corroboration used SymPy 1.14.0. No Lean executable was available, so all Lean compilation and kernel-replay statuses are `NOT_RUN`.

11.1 Main case construction and oracle separation

The 153 small group instances consist of all 36 ordered pairs of permutations on three points, 72 seeded random generator lists at degrees two through seven, and 45 named-family instances at degrees zero through eight. Repeated mathematical groups under different generator lists are intentional: the certificate must remain bound to the supplied generating data. They are not 153 nonisomorphic groups.

The independent small group oracle traverses a right Cayley graph. The chain producer uses incremental stabilizer construction. Coloring counts are compared with connected components of the full assignment graph, not with another Burnside implementation. Canonical images are compared with explicit images under the independently enumerated small group. These differing computations are stronger evidence than merely replaying the producer’s own residual claims.

Recorded category	Count	Interpretation
Small chain instances	153	Orders compared with complete small-group enumeration.
Membership queries	8,028	3,852 positive and 4,176 negative; exhaustive through degree five, sampled above it.
Generic canonical queries	296	Compared with the exact small-group minimum; 24,710 total cover nodes.
Small family-image queries	208	Exact symmetric/alternating shortcut compared with full group images.
Cycle inventories	40	Independently replayed through unique normal forms.
Color-count queries	160	Four color counts per inventory, compared with assignment orbits.
Fixed-weight queries	220	Compared with binary assignment orbits.
Small family-count comparisons	108	Family formula compared with the same small assignment-orbit oracle.
Large chain instances	7	No full enumeration; exact formula checks and separate SymPy corroboration.
Large family images	5	Explicit transporters and exact recognized-family minimum rule.
Malformed/corrupted controls	33	Rejected by the main suite.
CNF gates	5 / 10	Five accepted invariant inputs; ten rejected asymmetric variants.
Resource-refusal controls	5	All reported as unknown, not mathematical negatives.

These rows are different and sometimes overlapping units. They must not be added into a grand “proofs solved” or success-rate figure. For example, the same recognized family is tested through both a cycle inventory and a closed formula, and each chain may support many membership queries.

11.2 Large certificates

The order of $\mathfrak{S}_{40}$ in the recorded run is

$$815915283247897734345611269596115894272000000000 = 40!.$$

Group	DAG	Tables	Checks	Bytes	Search (s)	Check (s)
$\mathfrak{S}_{15}$	62	120	2,136	2,220	0.065	0.022
$\mathfrak{S}_{30}$	137	465	17,546	7,215	1.270	0.477
$\mathfrak{S}_{40}$	187	820	41,861	11,975	4.656	1.811
$\mathfrak{A}_{12}$	118	77	2,806	2,512	0.159	0.023
$\mathfrak{A}_{20}$	358	209	20,942	7,666	5.423	0.329
D_{50}	3	100	152	1,457	0.042	0.016
C_{97}	2	193	194	2,728	0.124	0.055

Table 2: Single-run measurements for the large chain fixtures. DAG counts retained word nodes; Tables counts transversal entries; Checks counts verified Schreier edges. Bytes is compact JSON for the chain alone, excluding the input and other certificates. Timings are descriptive, not comparisons against other systems.

Its chain certificate contains 187 word-DAG nodes and 820 transversal entries, serialized in 11,975 compact JSON bytes. The checker performed 41,861 Schreier checks. The degree is 40; the group order is not a count of explicitly visited permutations. The 820 entries describe the chain, and the separate canonical and polynomial certificates have their own sizes.

For the five recognized large symmetric/alternating cases, the corpus additionally stores 25 evaluations of the closed counting formulas. Those evaluations are replayed but are not 25 independent oracle comparisons. The small tests and the mathematical derivations provide different forms of evidence for the formulas.

11.3 Search-free replay

The main corpus contains 160 bundles: the 153 small cases and seven large cases. A standalone command runs the checker under `python -S`, which disables site-package loading. The replay script verifies that the producer module was not imported. It rechecks all chains, membership answers, image certificates, cycle inventories, stored counting evaluations, and corpus identities. Its result is `REPLAY_PASSED`.

The replay checks a nonempty manifest, exact filename coverage, duplicate-free manifest entries, and matching bundle names. It does not permit an empty directory to count as a successful run. Scripts explicitly refuse optimized Python execution with `-O`, because their regression assertions must remain enabled.

There are limits to this separation. Python itself, integer arithmetic, tuple representations, and ordinary programmer assumptions remain shared. The checker is not formally verified, and independently written implementations can have correlated mistakes. Search-free replay is evidence of modularity and resistance to fabricated search output; it is not kernel soundness.

11.4 The supplementary Reynolds run

The separate projection suite contains 25 small cases: five group families at each degree two through six. Each exact projection is compared with explicit averaging over every small group element. Each projected polynomial is then projected again and checked for idempotence. A twenty-sixth case is the $\mathfrak{S}_{40}$ mixed cubic with a 1,560-element monomial orbit.

Four malformed projection certificates are rejected: omitted source coverage, a false average coefficient,

an invalid generator edge, and a Boolean exponent. A separate `python -S` replay checks all 26 stored bundles without importing either producer. The main suite and the Reynolds suite are recorded separately, rather than silently changing the main run’s counts.

11.5 Regression tests, failures, and corrections

Twelve standard-library `unittest` methods pass. They include multiplication order, the empty action, incomplete stabilizer chains, incomplete orbits with a recomputed order product, malformed JSON, Boolean/integer confusion, alternating parity, the D_{10} count, semantic authorization, and resource-outcome separation. These are twelve test methods, not twelve theorem proofs.

The development history retains three noteworthy events. First, the generic $\mathfrak{S}_{15}$ image search hit its 20,000-node cap; the family-specific route was added rather than relabeling that failure. Second, manual decoder review found that an orbit-root comparison could accept `False` as the integer zero because of Python equality. The root check was changed to exact integer validation and explicit null tests, and a regression was added. Third, the first Reynolds mutation harness tried to mutate the second node of a singleton orbit and raised `IndexError`; it was corrected to select a nontrivial orbit and the full supplementary suite was rerun.

The JSON decoder rejects duplicate keys, floating-point mathematical inputs, non-finite constants, invalid permutations, unexpected fields, forward DAG references, and several size violations. These controls do not constitute hostile-input hardening. A production worker needs explicit total allocation, bit-length, recursion-depth, elapsed-time, and cancellation policies. Operating-system isolation remains advisable for external search.

11.6 Claims not supported by this evidence

No `forge` tactic was installed. No Lean certificate checker was implemented or compiled. No comparison with `grind`, `Aesop`, `LeanHammer`, `nlinarith`, `GAP`, or `nauty` was measured. No end-to-end source reifier was exercised. The cone-reduction adapter and constrained-coloring counter are designs, not delivered integrations. No generic polynomial-time canonicalizer or arbitrary-group cycle-inventory algorithm is claimed.

12 An implementation order with concrete acceptance gates

The first milestone should be a trusted group certificate, not a polished tactic command that delegates to unverified scripts. Each stage below has an acceptance condition that cannot be substituted by an easier near-miss.

Milestone 1: reflected group facts. Implement the permutation-array decoder, word-DAG evaluator, and complete-chain checker in Lean. Prove Theorem 3.3 and instantiate it on the complete small corpus. Accept only if both positive and negative membership claims are kernel checked against the exact original generator arrays. Showing that a permutation array is well formed is not this milestone. The rejected partial- $\mathfrak{S}_3$ chain must remain rejected after its order field is made internally consistent.

Milestone 2: one real source frontend. Bind the finite array model to an explicit source action on `Fin n`, with generator lemmas supplied by the user. Prove an original source theorem using the resulting group fact and ordinary Lean reasoning. Change one source generator and show that the old certificate no longer proves the old conclusion. A theorem only about the reified array object is not this milestone.

Milestone 3: representatives and counts. Formalize the cover checker and the symmetric/alternating shortcut. Then connect a checked cycle inventory to Mathlib’s orbit-counting theorem. Acceptance requires a theorem quantified over the color count, not a table of numerical evaluations. Include identity permutations with fixed points, degree zero, $q = 0$, and alternating all-distinct colorings in the tests.

Milestone 4: polynomial integration. Formalize the sparse orbit-average theorem and adapt the already selected Forge polynomial representation. Implement the dictionary-action gate and column-orbit reduction. Compare the expanded reduced certificate with the original cone checker, without changing the original target or weakening its hypotheses. Include a deliberately asymmetric hypothesis and a noninvariant target as rejection controls. A correct averaged polynomial alone is not a proof of the original inequality.

Milestone 5: automatic discovery and evaluation. Only after source-preserving explicit actions work, add graph-based symmetry proposals and external stabilizer-chain exporters. Evaluate on a corpus that contains genuinely symmetric goals, asymmetric near-misses, already easy `grind` goals, and goals where the symmetry machinery has overhead but no benefit. Pin Lean, Mathlib, imported modules, tactic options, timeouts, and machine details. Report source goals solved, proof-check time, certificate size, resource refusals, and axiom inventories separately. Run ablations with chain caching, canonical covers, family shortcuts, and dictionary reduction individually disabled.

A fair comparison must ask the original tactics and the extended tactic to prove the *same source statements*. Replacing a group-order goal by a supplied order fact, or supplying the sought canonical image as an assumption, would test a different task. The existing Forge run counts are not baselines for these new instances and should not be pooled with them.

13 Further research that the certificate boundary makes possible

The delivered construction gives several precise next problems rather than an unbounded wish list.

13.1 Shorter bases and proof compression

The full base is robust but often redundant. A short-base format should supply a checked certificate that the terminal stabilizer is trivial. One route is a second faithful action on a smaller derived domain; another is a terminal generator list checked to contain only the identity. Any such format must still justify that the listed terminal generators generate the actual stabilizer. Merely observing that the current list is empty repeats the incomplete-chain error.

Word-DAG compression can also be made cost aware. Several certificates use few strong roots but many intermediate values. Search can minimize retained nodes after trimming, rather than minimize

its own total generated nodes. A suitable acceptance metric is checked proof size at fixed degree and generator input, not the apparent length of an unexpanded word.

13.2 Better canonical bounds with local certificates

The coordinatewise orbit-minimum relaxation ignores the impossibility of reusing the same low color everywhere. A stronger bound can track color multiplicities within a block and solve a minimum-cost matching relaxation. Its dual or exact combinatorial witness can be attached to a coset cut. The proof obligation is a lower bound on *every* image in the represented coset; a plausible completion found by search is an upper bound and cannot authorize pruning.

The coset-cover design permits this replacement locally. Old bounds remain admissible, stronger bounds can be tried opportunistically, and the covering theorem is unchanged. Whether the additional checker cost pays for fewer cover nodes is an empirical question for a source-level benchmark.

13.3 Conjugacy-class inventories with coverage proofs

The current general counter enumerates every group element. A more scalable inventory could use representatives with exact centralizer orders and verified conjugacy-class coverage. Verifying only the class sizes sum to $|G|$ is insufficient unless disjointness and membership are established: duplicated classes could have the right total. A usable certificate needs an exact class partition or another theorem establishing completeness, plus source-owned words for representatives. This is a natural next finite-group worker, but none of these class certificates is implemented here.

13.4 Representation-theoretic algebraic compression

The orbit-reduced cone theorem handles a permutation action on a finite dictionary. It does not diagonalize general group representations or decompose semidefinite Gram matrices. A later worker could export rational invariant subspaces, change-of-basis matrices, and exact block-commutation identities, then reuse Forge’s existing algebraic checkers. The established symmetry/SOS literature provides the algorithmic direction [14]; the new work would be a compact Lean-checkable decomposition certificate and an exact reconstruction of the original polynomial claim.

14 Conclusion

The principal recommendation is to make certified finite symmetry a reusable Forge service. Its first theorem should turn a complete, source-bound stabilizer-chain certificate into exact group facts. Its first consumers should be checked canonical images and orbit counts. Sparse Reynolds projection then connects that service to the polynomial engines Forge already has, while generator-wise semantic authorization connects it to proof search without treating renaming as arbitrary equality.

The prototype demonstrates that this is an implementable proposal rather than a slogan: exact constructors, separate local checkers, complete stored certificates, independent small oracles, large non-enumerative group examples, negative controls, and search-free replay are supplied. The remaining step is substantial and explicit: formalize the checkers and bind them to Lean source obligations. Nothing in the measured Python results removes that obligation. It does, however, give a sharply bounded implementation target with useful capabilities beyond the inspected Forge catalogue.

A Using the delivered package

The executable core requires only Python’s standard library. It was tested on Python 3.13.5. No installation or network connection is required for the core commands. Run them from the `prototype/` directory:

```
python -S demo.py
python -S -m unittest discover -s tests -v
python -S verify_corpus.py ../results/recorded
python -S verify_reynolds.py ../results/reynolds/bundles.json
```

The first command prints the D_{10} bracelet counts, the alternating canonical-image example, and a four-variable Reynolds projection. The replay commands consume recorded certificates without importing search code.

To regenerate independent experiment outputs without overwriting the recorded run:

```
python -S run_experiments.py --output ../reproduced-main
python -S reynolds_experiments.py --output ../reproduced-reynolds
```

The destination must not already exist. The optional `-skip-large` flag shortens the main construction suite but does not remove its fixed $\mathfrak{S}_{15}$ resource-refusal control. It is not the same experiment as the recorded full run. Do not use `-O`: the scripts refuse to disable their assertions.

The optional external corroboration command is

```
python optional_sympy_oracle.py
```

It requires SymPy; the supplied result was obtained with version 1.14.0. It is deliberately separate from all `-S` commands and from the package requirements of the actual certificate checker.

A.1 A direct API example

```
from forge_symmetry.producer import named_problem, build_chain
from forge_symmetry.checker import verify_chain, CheckLimit

problem = named_problem("S", 7)
certificate = build_chain(problem).export()
checked = verify_chain(problem, certificate)
print(checked.order)           # 5040
print(checked.contains([1,0,2,3,4,5,6])) # True

# A resource cap is not a negative mathematical answer.
try:
    verify_chain(problem, certificate, max_checks=0)
except CheckLimit:
    print("unknown: checking budget exhausted")
```

A production caller must retain the original problem independently of the returned certificate. The test bundles store both for reproducibility, but that co-location is not a source-reification proof.

A.2 Source and result inventory

Path	Purpose
article/	Complete editable LaTeX source, compiled PDF, and build script.
prototype/forge_symmetry/	Group/image/counting producer and checker; separate Reynolds producer and checker.
prototype/tests/	Small regression suite.
results/recorded/	Main manifest, 160 corpus bundles, metrics, rejection controls, resource refusals.
results/reynolds/	Separate 26-case projection corpus and result summary.
results/replay.json	Standalone main replay receipt.
results/reynolds_replay.json	Standalone projection replay receipt.
results/sympy_oracle.json	Optional external order and membership corroboration.
results/development_history.json	Preserved limitations and corrected implementation/test issues.
docs/	Scoped novelty audit, status ledger, protocol notes, and source inventory.
lean/IMPLEMENTATION.md	Formalization worklist; explicitly no implemented Lean tactic or checker.

A.3 Building the article

From `article/`, run `sh build.sh`, or invoke `pdfLaTeX` three times:

```
pdflatex -interaction=nonstopmode -halt-on-error forge-certified-symmetry.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-certified-symmetry.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-certified-symmetry.tex
```

The document uses ordinary TeX Live packages and needs neither shell escape nor a bibliography program. No third-party source tree or font file is redistributed in the archive.

B Status at a glance

Capability	Status in this artifact
Exact chain construction and separate checking	Implemented and executed in Python.
Membership and nonmembership queries	Implemented; independent small-oracle tests.
Generic canonical-image covers	Implemented; bounded and sometimes refused.
Symmetric/alternating image shortcuts	Implemented; exact recognition gates and tests.
General small-group cycle inventories	Implemented; full normal-form replay under a cap.
Symbolic family counting rules	Implemented as exact evaluators; mathematical universal formulas proved in the article.
Sparse Reynolds projection	Implemented; separate local checker, differential tests, replay.
Concrete CNF/objective symmetry gate	Implemented for unsigned variable permutations.
Orbit-reduced cone adapter	Designed; not integrated into Forge.
Automatic graph-based source discovery	Designed; not implemented.
Reflected Lean checkers and source reifier	Not implemented; no Lean compilation performed.
Lean tactic evaluation or measured solved-goal gain	Not performed.

References

- [1] Vladimir Reshetnikov and contributors. *Forge: repository README and three-round contribution catalogue*. Revision 58ea206bd7ad501b930add568151217bcc7f82a2, inspected September 15, 2026. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/README.md>.
- [2] Vladimir Reshetnikov and contributors. *Forge implementation status*. Same pinned revision as [1]. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/docs/STATUS.md>.
- [3] Vladimir Reshetnikov and contributors. *Forge certificate schemas*, article appendix source. Same pinned revision as [1]. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/article/sections/A-schemas.tex>.
- [4] Vladimir Reshetnikov and contributors. *Ideas from Leant and Djex*. Same pinned revision as [1]. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/docs/IDEAS-FROM-LEANT-DJEX.md>.
- [5] Vladimir Reshetnikov and contributors. *Leant: a Djex-based synthesis REPL for Lean 4*, README. Revision 6bf05ad78c467989e68290f2d08bbbed40802d485, inspected September 15, 2026. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/README.md>.
- [6] Vladimir Reshetnikov and contributors. *Djex*, README. Revision e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5, inspected September 15, 2026. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/README.md>.
- [7] The GAP Group. *GAP Reference Manual, Chapter 43: Permutation Groups*; and permutation-group capabilities. Official documentation, accessed September 15, 2026. GAP was not executed for this artifact. https://docs.gap-system.org/doc/ref/chap43_mj.html; <https://www.gap-system.org/capabilities/permggroups>.
- [8] SymPy Development Team. *Permutation Groups*, SymPy 1.14.0 documentation. Official API documentation, accessed September 15, 2026. https://docs.sympy.org/latest/modules/combinatorics/perm_groups.html.
- [9] The Mathlib Community. *Mathlib.GroupTheory.GroupAction.Quotient*. In particular `MulAction.sum_card_fixedBy_eq_card_orbits_mul_card_group`. Generated documentation, accessed September 15, 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/GroupTheory/GroupAction/Quotient.html.
- [10] The Mathlib Community. *Mathlib.GroupTheory.Perm.Cycle.Type*. In particular `cycleType`, `two_le_of_mem_cycleType`, `sum_cycleType`, and `card_fixedPoints`. Generated documentation, accessed September 15, 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/GroupTheory/Perm/Cycle/Type.html.
- [11] Brendan D. McKay and Adolfo Piperno. Practical graph isomorphism, II. *Journal of Symbolic Computation* 60 (2014), 94–112. DOI: 10.1016/j.jsc.2013.09.003. <https://arxiv.org/abs/1301.1493>. Official software: <https://pallini.di.uniroma1.it/>.

- [12] Bart Bogaerts, Stephan Gocht, Ciaran McCreesh, and Jakob Nordström. Certified Symmetry and Dominance Breaking for Combinatorial Optimisation. *Journal of Artificial Intelligence Research* 77 (2023), 1539–1589. DOI: [10.1613/jair.1.14296](https://arxiv.org/abs/2203.12275). <https://arxiv.org/abs/2203.12275>.
- [13] Markus Anders, Bart Bogaerts, Benjamin Bogø, Arthur Gontier, Wietze Koops, Ciaran McCreesh, Magnus O. Myreen, Jakob Nordström, Andy Oertel, Adrian Rebola-Pardo, and Yong Kiam Tan. *Faster Certified Symmetry Breaking Using Orders With Auxiliary Variables*. arXiv:2511.16637, 2025; inspected version 1. <https://arxiv.org/abs/2511.16637>.
- [14] Karin Gatermann and Pablo A. Parrilo. Symmetry groups, semidefinite programs, and sums of squares. *Journal of Pure and Applied Algebra* 192, nos. 1–3 (2004), 95–128. DOI: [10.1016/j.jpaa.2003.12.011](https://arxiv.org/abs/math/0211450). <https://arxiv.org/abs/math/0211450>.